

Fișă de lucru 1 - Python: Mutabilitate și Subprograme

1. Teorie și exemple

În Python, tipurile de date se împart în două mari categorii, în funcție de capacitatea lor de a fi modificate după ce au fost create:

A. Tipuri IMUTABLE (nu pot fi modificate pe loc)

Aici intră numerele (`int`, `float`), șirurile de caractere (`string`) și tuplurile. Dacă încerci să le modifice, Python va crea de fapt o *copie nouă* în memorie.

- **Cum se comportă în funcții:** Când trimiți o variabilă imutabilă ca parametru, modificările din interiorul subprogramului **NU** afectează variabila originală din exterior.

Exemplu Rezolvat:

```
def adauga_zece(n):
    n = n + 10 # Se modifică doar copia locală a lui n
    print("În interior, n este:", n)

x = 5
adauga_zece(x)
print("În exterior, x este:", x) # Va afișa 5
```

B.

Tipuri MUTABLE (pot fi modificate pe loc)

Aici intră **listele** și dicționarele. Dacă modifice o listă (schimbi un element, adaugi cu `append`), modificarea se face direct pe obiectul original din memorie.

- **Cum se comportă în funcții:** Când trimiți o listă ca parametru, modificările (in-place) se vor vedea **și în variabila originală** (la fel ca transmiterea vectorilor ca parametri în C++).

Exemplu Rezolvat:

```
def adauga_element(lista_primita):
    lista_primita.append(4) # Modificăm direct lista în memorie

lista_mea = [1, 2, 3]
adauga_element(lista_mea)
print("În exterior, lista este:", lista_mea) # Va afișa [1, 2, 3, 4]
```

2. Exerciții de înțelegere (Tracing)

Ce se va afișa pe ecran în urma execuției următoarelor blocuri de cod? Încercuiți varianta corectă sau completați spațiul punctat.

Exercițiul 1.

```
def functie_misterioasa(a):  
    a = a + 2  
  
x = 7  
functie_misterioasa(x)  
print(x)
```

- A) 7
- B) 9
- C) Eroare, funcția nu returnează nimic.

Exercițiul 2.

```
def schimba_mesaj(text):  
    text = text + "!"  
  
salut = "Salut"  
schimba_mesaj(salut)  
print(salut)
```

- A) Salut!
- B) Salut
- C) Eroare

Exercițiul 3.

```
v = [10, 20, 30]

def schimba_primul(vector):
    vector[0] = 0

schimba_primul(v)
print(v[0])
```

Răspuns liber (dați și o scurtă explicație):

Exercițiul 4.

```
nr = 5
def dublare(x):
    x = x * 2
    return x

dublare(nr)
print(nr)
```

Răspuns liber (dați și o scurtă explicație):

Bibliografie:

<https://www.geeksforgeeks.org/python/mutable-vs-immutable-objects-in-python/>

<https://realpython.com/python-mutable-vs-immutable-types/>